AWE NUCLEAR SECURITY TECHNOLOGIES

EPOCH Workshop 2026: Particles, currents, pushers

Stuart Morris (AWE Nuclear Security Technologies)

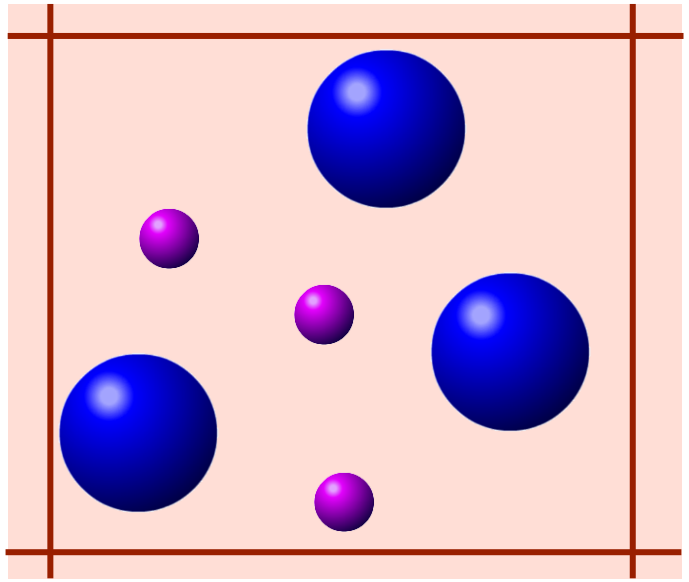
Chris Ridgers, Joel Adams, Sam O'Neill, Nigel Woolsey (University of York)

Tom Goffrey (University of Warwick)

Macro-particles

```
begin:constant
  drift_p = 2.5e-24
  temp = 273
  dens = 10
end:constant


begin:species
  # Rightwards travelling electrons
  name = Right
  charge = -1
  mass = 1.0
  temp = temp
  drift_x = drift_p
  number_density = dens
  npart = 4 * nx
end:species
```

- Load macro-particles from species block
  - Macro-particles loaded at random positions in a cell
  - Number of marco-particles in cell, N_{ppc}: npart or frac
- Weight: $W_e = n_e V / N_{ppc}$ for density, n_e, cell volume, V
  - In 1D/2D, missing dimensions of cell-size 1m
  - This is fine, EPOCH equations only sensitive to density
- Momentum distribution:
  - Default: Maxwellian at cell temperature
  - User-given drift momentum applied
  - Custom distributions possible

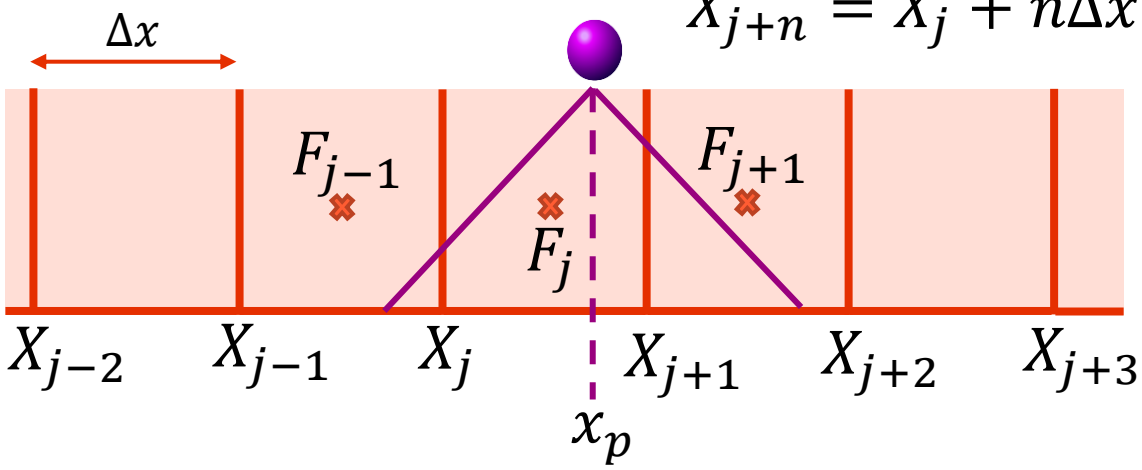


UK Ministry of Defence © Crown owned copyright 2026/AWE

1D shape functions

$$X_j = \text{floor}\left(\frac{x_p}{\Delta x}\right)\Delta x$$

$$X_{j+n} = X_j + n\Delta x$$



- Macro-particles have weight, W_e with "shape", $S(x)$.

$$S(x) = \begin{cases} 1 - \dfrac{|x - x_p|}{\Delta x} & |x - x_p| < \Delta x \\ 0 & \text{otherwise} \end{cases}$$

- Calculate weight in each cell using f:

$$f = \frac{X_j}{\Delta x} - \left(\frac{x_p}{\Delta x} - \frac{1}{2}\right)$$

$$W_{j-1} = \frac{1}{2}W_e\left(\frac{1}{4} + f^2 + f\right)$$
$$W_j = \frac{1}{2}W_e\left(\frac{3}{2} - 2f^2\right)$$
$$W_{j+1} = \frac{1}{2}W_e\left(\frac{1}{4} + f^2 - f\right)$$

- Sanity check: if particle position x_p is on a boundary between two cells: $x_p = X_j$

$$f = \frac{1}{2}, \qquad W_j = W_{j-1} = \frac{1}{2}$$

- For cell-centred variable F evaluated at the particle position:

$$F = W_{j-1}F_{j-1} + W_jF_j + W_{j+1}F_{j+1}$$

UK Ministry of Defence © Crown owned copyright 2026/AWE

Alternative shape functions

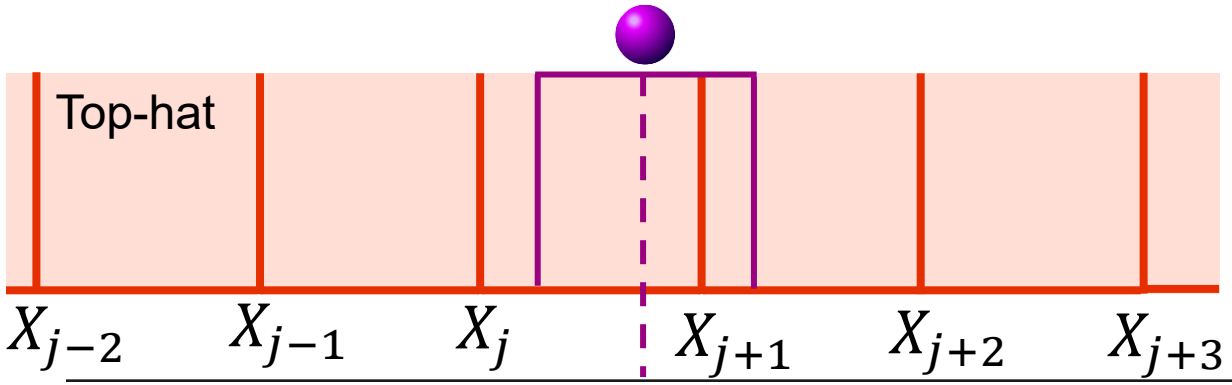
- Macro-particles can have one of three shapes:
  - Top-hat (shape size: Δx)
  - Default (shape size: $2\Delta x$)
  - B-Spline (shape size: $4\Delta x$)

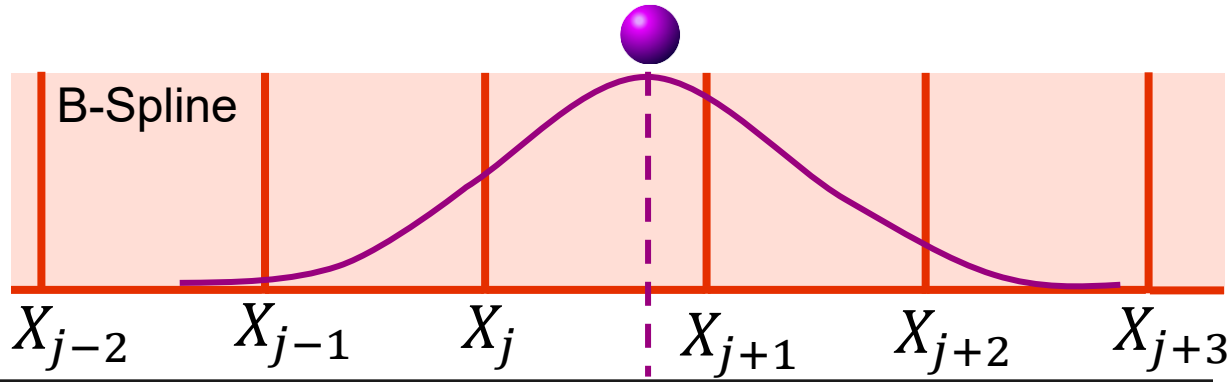
$$S(x) = \begin{cases} 1/\Delta x & |x - x_p| < \Delta x/2 \\ 0 & \text{otherwise} \end{cases}$$

$$S(x) = \begin{cases} 1 - \dfrac{|x - x_p|}{\Delta x} & |x - x_p| < \Delta x \\ 0 & \text{otherwise} \end{cases}$$

$$S(x) = \frac{1}{6\Delta x^4} \begin{cases} \left(-|x - x_p| + 2\Delta x\right)^3 & x \leq |x - x_p| \leq 2\Delta x \\ 3|x - x_p|^3 - 6\Delta x|x - x_p|^2 + 4\Delta x^3 & |x - x_p| < \Delta x \\ 0 & \text{otherwise} \end{cases}$$

- Large particle shapes have smoother currents
- Larger shapes also take longer to compute



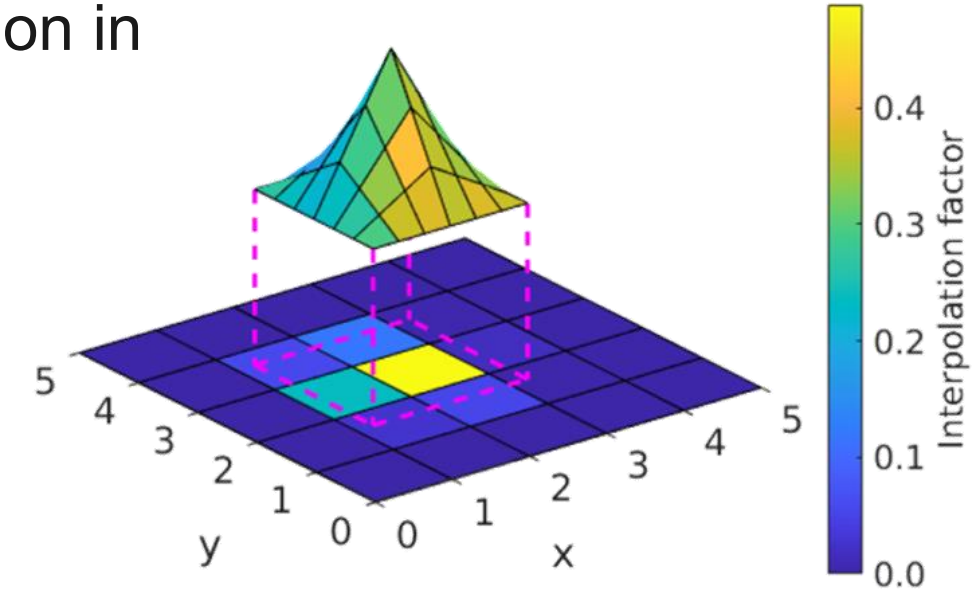


UK Ministry of Defence © Crown owned copyright 2026/AWE

General shape functions

- Macro-particle shapes in 2D/3D
  - Calculate shape function in each dimension
  - Multiply 1D weighting factors for each dimension in each cell

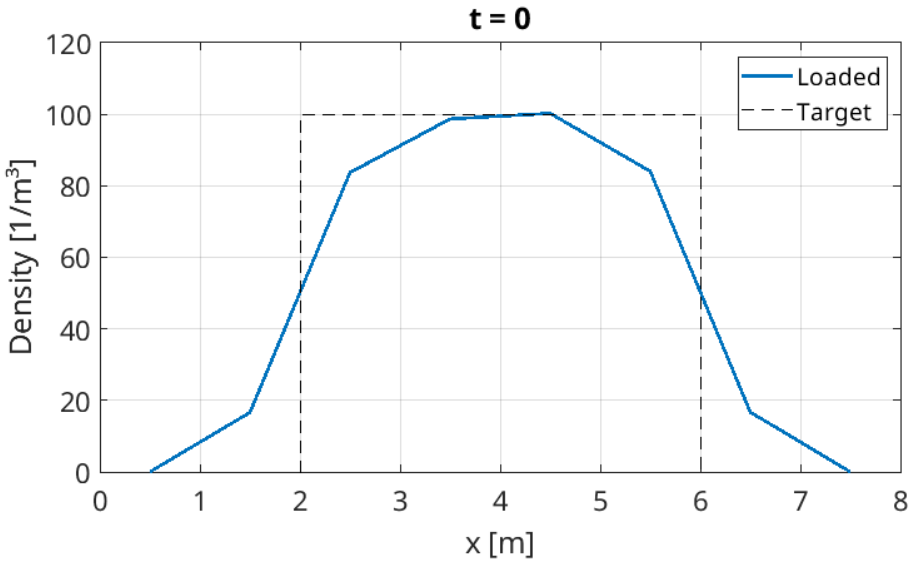
- 2D triangular weighting example:



UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 3.1: Particle shape effects

- Load particles into a few cells in 1D, output initial density
  - Aim for spatial step-function in density
  - See density in cells without particle initialisation




```
begin:control
  nx = 8
  nsteps = 0
  x_min = 0
  x_max = 8
end:control

begin:boundaries
  bc_x_min = open
  bc_x_max = open
end:boundaries

begin:species
  name = Electron
  charge = -1.0
  mass = 1.0
  nparticles = nx * 1000
  density = if ((x gt 2) and (x lt 6), 100, 0)
end:species

begin:output
  number_density = always
end:output
```


UK Ministry of Defence © Crown owned copyright 2026/AWE

Leap-frog pusher

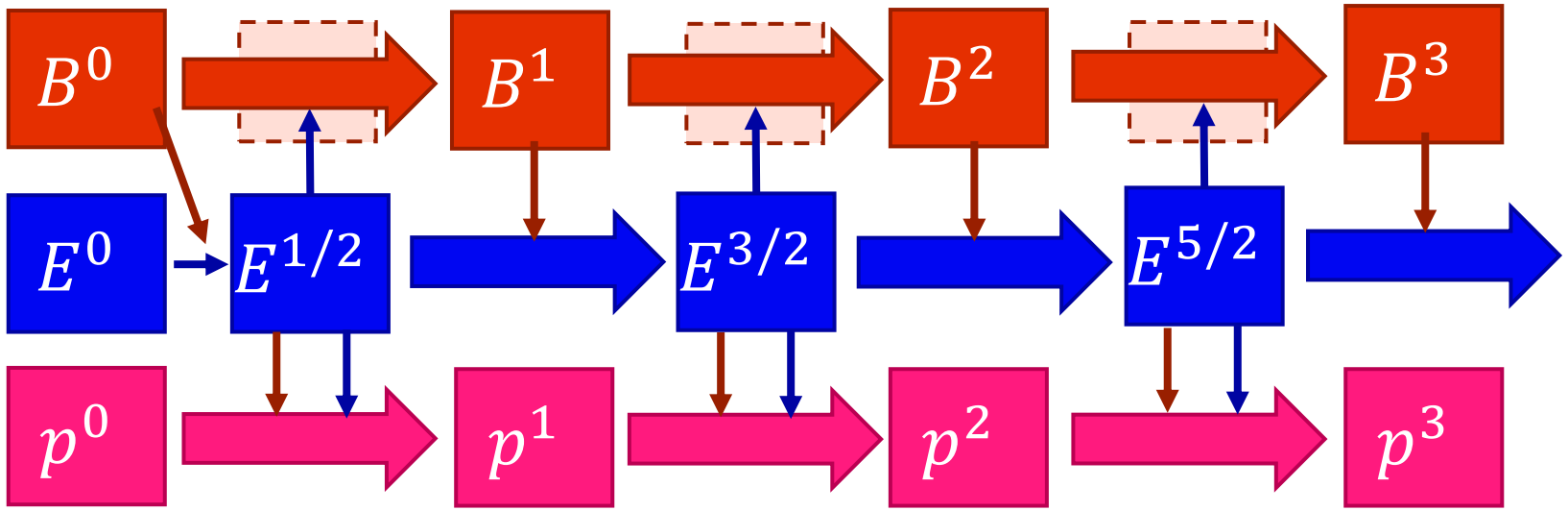
- Lorentz force:
  - $\frac{d}{dt}\boldsymbol{p} = q(\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B})$
- Discretised:
  - $\boldsymbol{p}^{n+1} = \boldsymbol{p}^{n} + q\Delta t\left(\boldsymbol{E}^{n+\frac{1}{2}} + \frac{1}{\gamma^{n+\frac{1}{2}}m}\boldsymbol{p}^{n+\frac{1}{2}} \times \boldsymbol{B}^{n+\frac{1}{2}}\right)$

$\boldsymbol{E}^{n+\frac{1}{2}}$: from field solve

$\boldsymbol{B}^{n+\frac{1}{2}}$: using two half-steps

$\boldsymbol{p}^{n+\frac{1}{2}}$: average $\frac{1}{2}(\boldsymbol{p}^{n+1} + \boldsymbol{p}^{n})$

$\gamma^{n+\frac{1}{2}}$: special treatment



UK Ministry of Defence © Crown owned copyright 2026/AWE

Momentum update

- Solve Lorentz force update using time-centred fields

$$\boldsymbol{p}^{n+1} = \boldsymbol{p}^{n} + q\Delta t\left(\boldsymbol{E}^{n+\frac{1}{2}} + \frac{1}{2\gamma^{n+\frac{1}{2}}m}(\boldsymbol{p}^{n+1}+\boldsymbol{p}^{n}) \times \boldsymbol{B}^{n+\frac{1}{2}}\right)$$

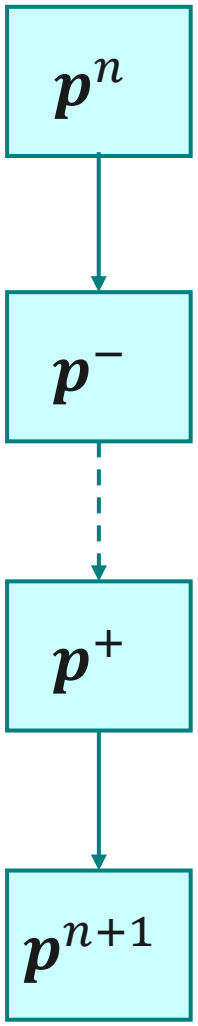
- Introduce $\boldsymbol{p}^{-}$, $\boldsymbol{p}^{+}$ and split update into acceleration and rotation:

$$\boldsymbol{p}^{-} = \boldsymbol{p}^{n} + \frac{\Delta t}{2} q\boldsymbol{E}^{n+\frac{1}{2}}$$

$$\boldsymbol{p}^{n+1} = \boldsymbol{p}^{+} + \frac{\Delta t}{2} q\boldsymbol{E}^{n+\frac{1}{2}}$$

- Let us use $\boldsymbol{p}^{-}$ for $\gamma^{n+\frac{1}{2}} = \sqrt{1 + \boldsymbol{p}^{-} \cdot \boldsymbol{p}^{-}/m^2c^2}$
- Substitute $\boldsymbol{p}^{-}$, $\boldsymbol{p}^{+}$ into Lorentz force – no E, but not obvious how to solve

$$\boldsymbol{p}^{+} = \boldsymbol{p}^{-} + \frac{q\Delta t}{2\gamma^{n+\frac{1}{2}}m}(\boldsymbol{p}^{+}+\boldsymbol{p}^{-}) \times \boldsymbol{B}^{n+\frac{1}{2}}$$

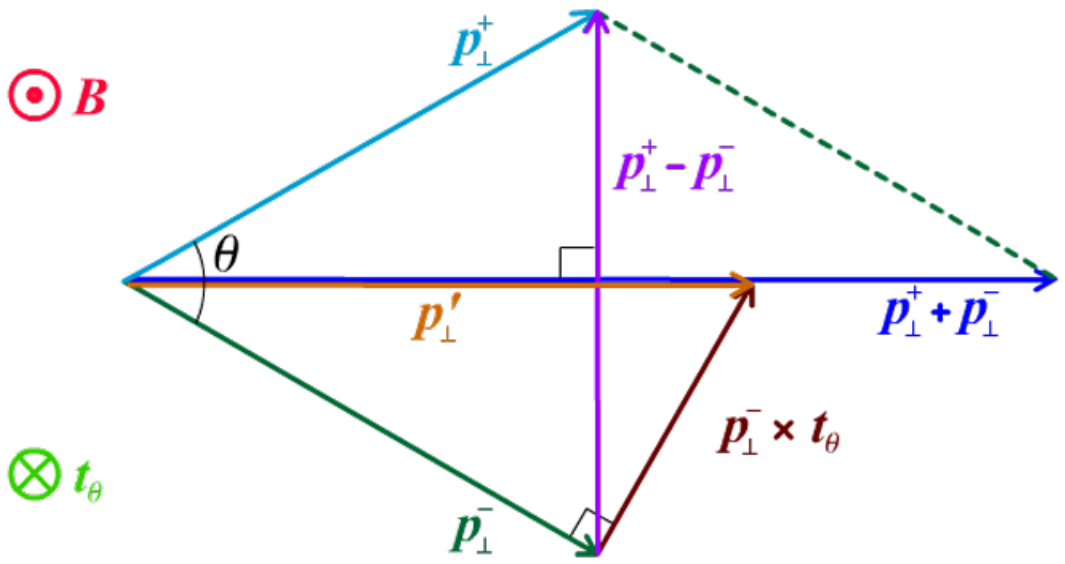


UK Ministry of Defence © Crown owned copyright 2026/AWE

$$\boldsymbol{p}^{+} = \boldsymbol{p}^{-} + \frac{q\Delta t}{2\gamma^{n+\frac{1}{2}}m}(\boldsymbol{p}^{+}+\boldsymbol{p}^{-}) \times \boldsymbol{B}^{n+\frac{1}{2}}$$

Boris pusher

- For component parallel to $\boldsymbol{B}$, cross-product is 0, so $p_{\parallel}^{+} = p_{\parallel}^{-}$
- Define two new vectors:
  - $\boldsymbol{t}_{\theta} = \frac{q\Delta t}{2\gamma^{n+\frac{1}{2}}m} \boldsymbol{B}^{n+\frac{1}{2}}$
  - $\boldsymbol{p}' = \boldsymbol{p}^{-} + \boldsymbol{p}^{-} \times \boldsymbol{t}_{\theta}$
- It can be shown that:
  - $\boldsymbol{p}^{+} - \boldsymbol{p}^{-} = \frac{2}{1+t_{\theta}^{2}}(\boldsymbol{p}' \times \boldsymbol{t}_{\theta})$
  - This links p^{n} to p^{n+1}



UK Ministry of Defence © Crown owned copyright 2026/AWE

Macro-particle current (grid-direction)

- Continuity Equation:

$$\frac{\partial \rho}{\partial t} = -\nabla \cdot \boldsymbol{J}$$

- Particle shapes independent in each dimension:

$$\frac{\Delta \rho_x}{\Delta t} = -\frac{\Delta J_x}{\Delta x}$$

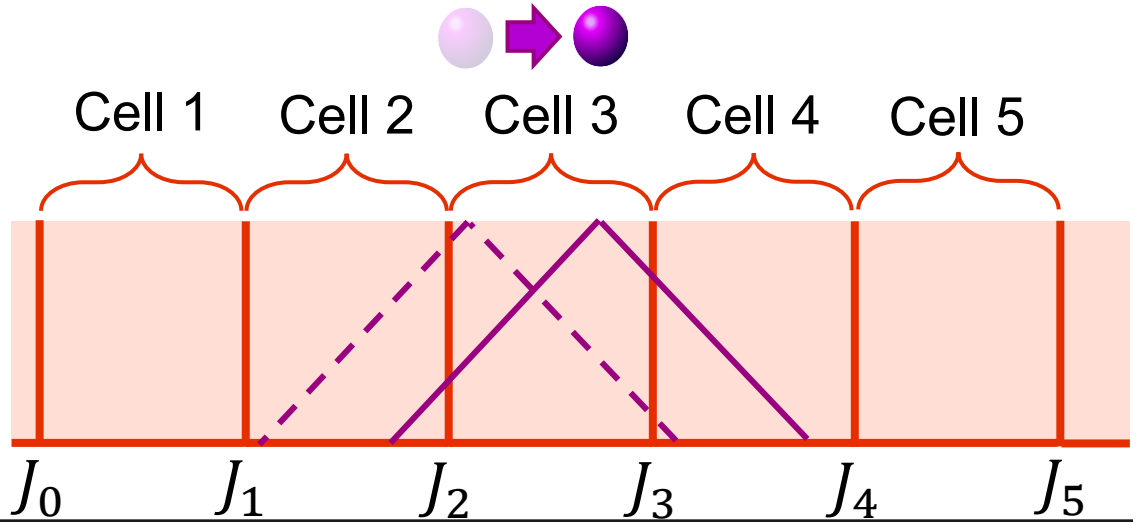
- E.g.: Cell 2 weight change: −0.3

$$J_2 - J_1 = \frac{\Delta x}{\Delta t}\left(-\frac{0.3 W_e q_e}{V}\right)$$

  - No charge passes J_1 line, so $J_1 = 0$
  - Next, $J_3 = J_2 + \frac{\Delta x}{\Delta t}\left(+0.15\frac{W_e q_e}{V}\right)$
  - Repeat until all boundaries considered

W_e: macro-particle weight
q_e: particle charge
V: cell volume
Δx: cell-size
Δt: time-step

Cell	Weight before	Weight after
1	0	0
2	0.40	0.10
3	0.55	0.70
4	0.05	0.20
5	0	0



UK Ministry of Defence © Crown owned copyright 2026/AWE

Macro-particle current (omitted-direction)

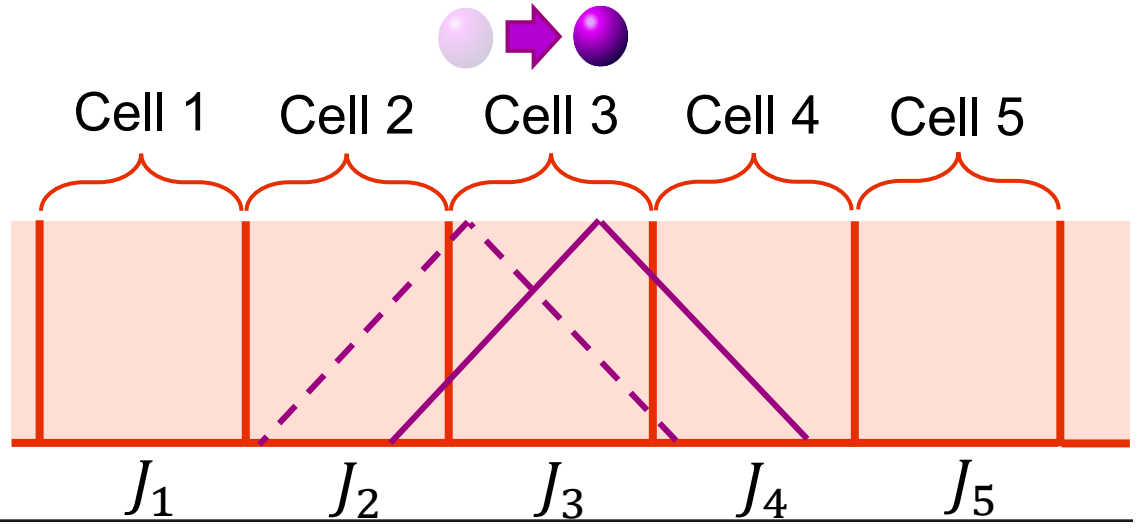
- Missing directions in EPOCH1D/2D have:
  - J: cell-centred variable
  - Calculate using average-weight in cell
  - In EPOCH1D: $J_y = \frac{W_e q_e}{V} v_y \langle g \rangle$
  - E.g. cell 2: $\langle g \rangle = 0.25$
- For J_z in EPOCH2D, average over x, y:

$$\langle g \rangle = g_x g_y + \frac{1}{2} g_x \Delta g_y + \frac{1}{2} g_y \Delta g x + \frac{1}{3} \Delta g_x \Delta g_y$$

  - g_i is fraction of weight in cell, direction i

W_e: macro-particle weight
q_e: particle charge
V: cell volume
v: particle velocity
$\langle g \rangle$: average weight frac.

Cell	Weight before	Weight after
1	0	0
2	0.40	0.10
3	0.55	0.70
4	0.05	0.20
5	0	0

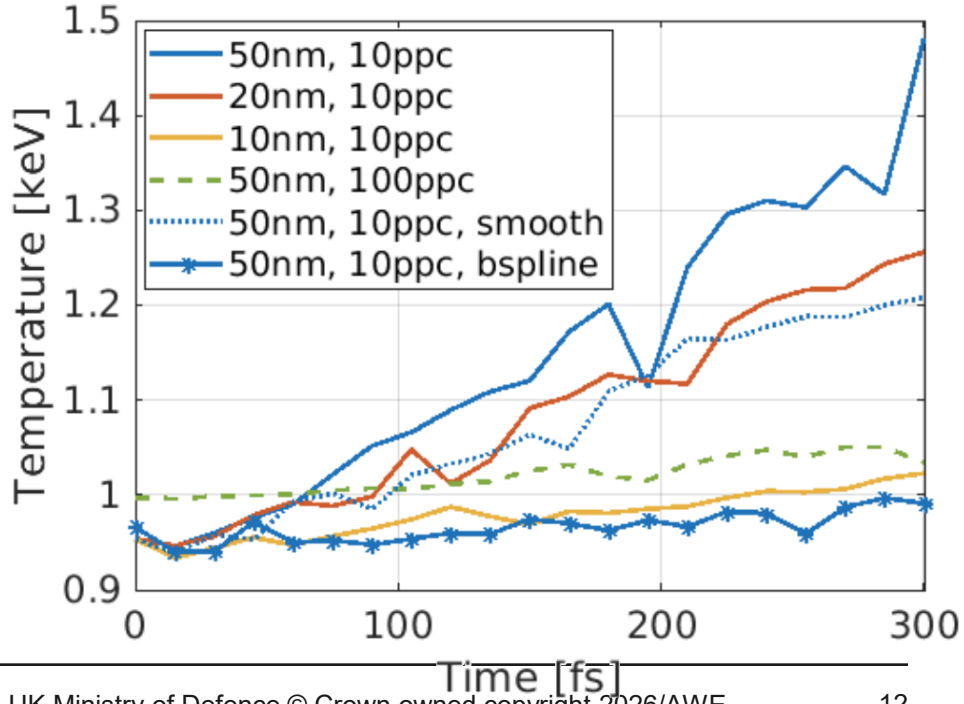


UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 3.2: Self-heating

```
begin:constant
    cell_size = 50.0e-9
    parts_per_cell = 10
end:constant

begin:control
    nx = 10
    ny = 10
    t_end = 300.0e-15
    x_min = 0
    x_max = nx * cell_size
    y_min = 0
    y_max = ny * cell_size
    stdout_frequency = 100
    smooth_currents = T
end:control
```

- At low resolution, macro-particle motion causes large currents
  - Non-physical, we should have $> 10^{20}$ particles, with smooth distribution
- Noisy currents give noisy E field
  - Large noise-driven gradients generate large B
  - Noisy fields cause non-physical particle heating
  - Worse in colder, denser plasmas
- Currents can be smoothed by using:
  - Larger particle shapes
  - Averaging currents across cells: `smooth_currents = T`
  - More particles per cell
  - Smaller cells
- Test with small, periodic domain

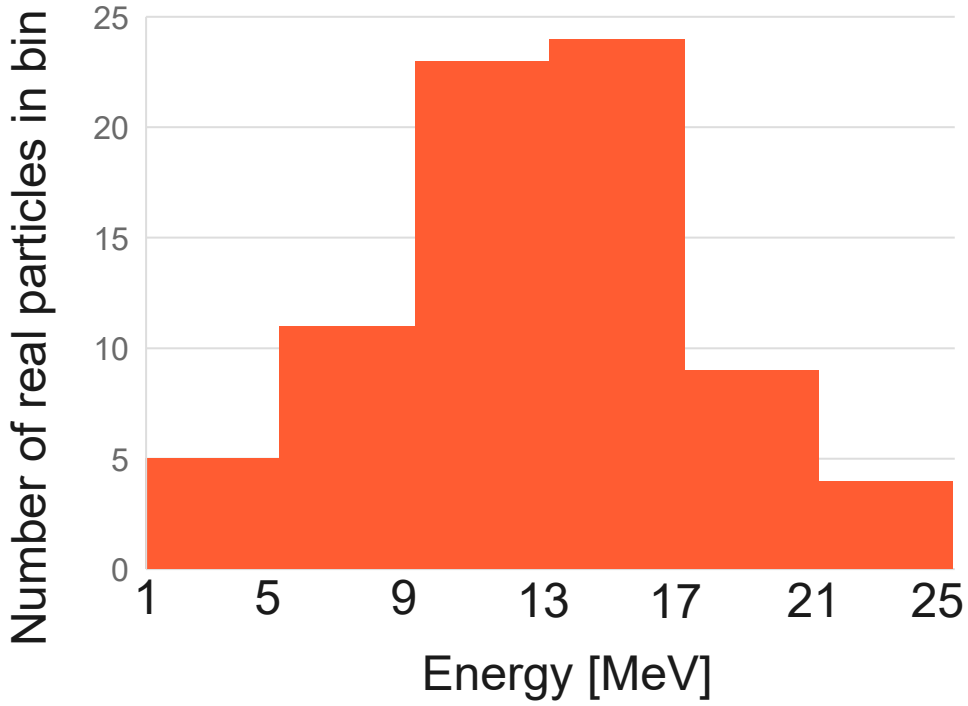


UK Ministry of Defence © Crown owned copyright 2026/AWE

Particle outputs

- Macro-particles have positions, momenta, and weights which we can output to make spectra
  - Each macro-particle has energy: $E = \sqrt{(p_x^2 + p_y^2 + p_z^2)c^2 + m^2c^4}$
  - Obtain energy bin limits
  - Find out which bin each particle belongs to
  - Sum the weights in each bin

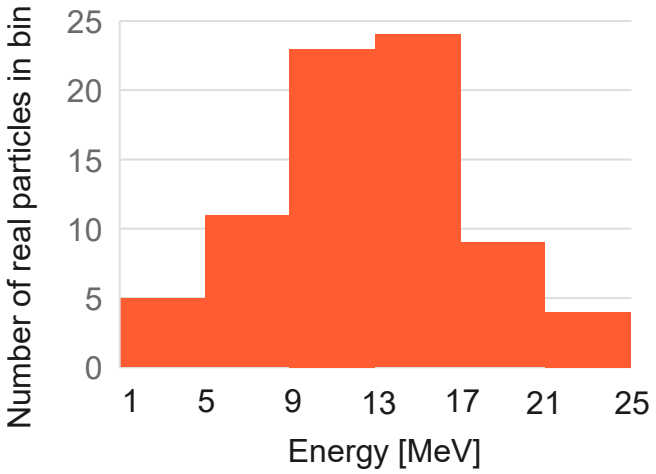
- Useful functions for binning:
  - Python: np.histogram(energy, bins=edges, weights=w)
  - MATLAB: histcounts(energy)



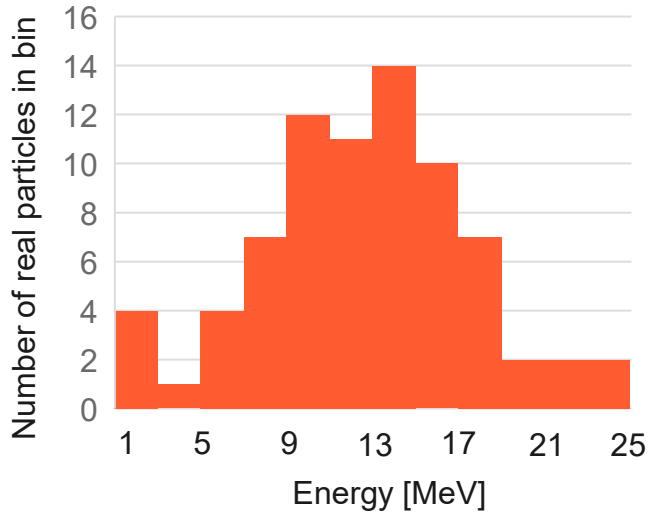
UK Ministry of Defence © Crown owned copyright 2026/AWE

Particle binning issues

- Changing bin number can drastically change how spectra appear
- Vertical scale is a function of bin-size
  - 6 bins: peaks at 24
  - 12 bins: peaks at 14
- Bin-size is arbitrary – we don't want our results to depend on this!



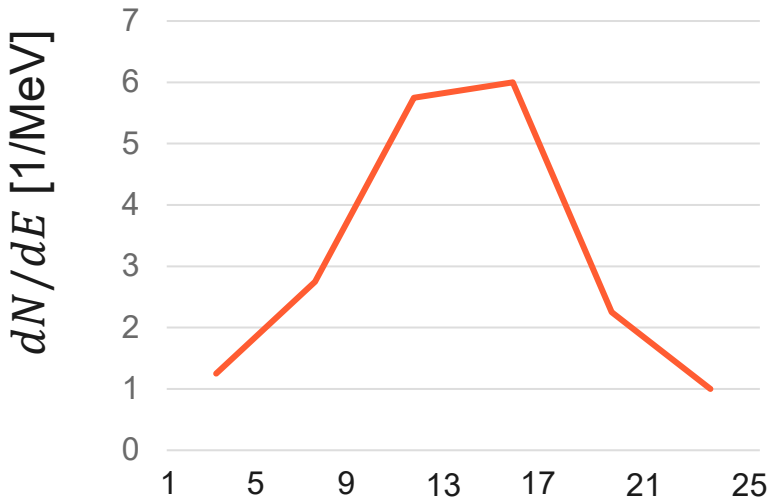
Same data!

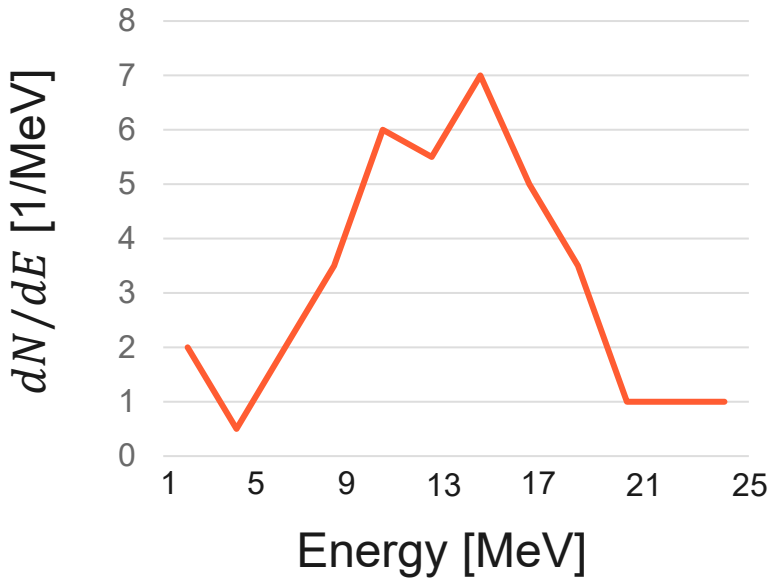


UK Ministry of Defence © Crown owned copyright 2026/AWE

Differential spectra

- Dividing bin-counts by the bin-widths:
  - Number of real particles between two limits is now area under plot between them
  - Scaling is now independent of bin-size – both plots peak around 6/MeV
  - Since integral is a particle number, N, vertical axis must be differential dN/dE



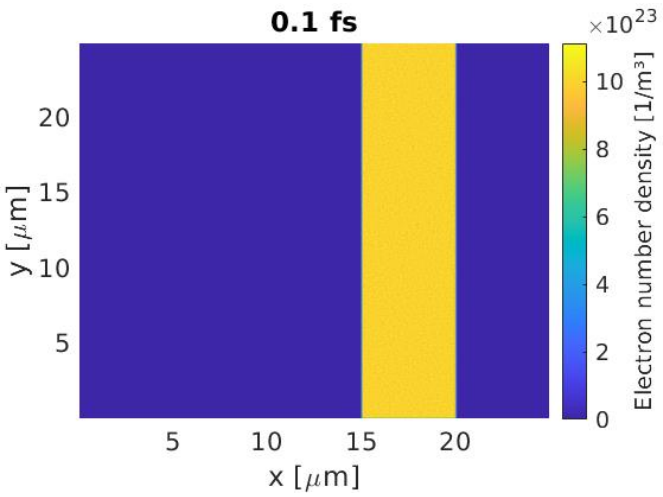


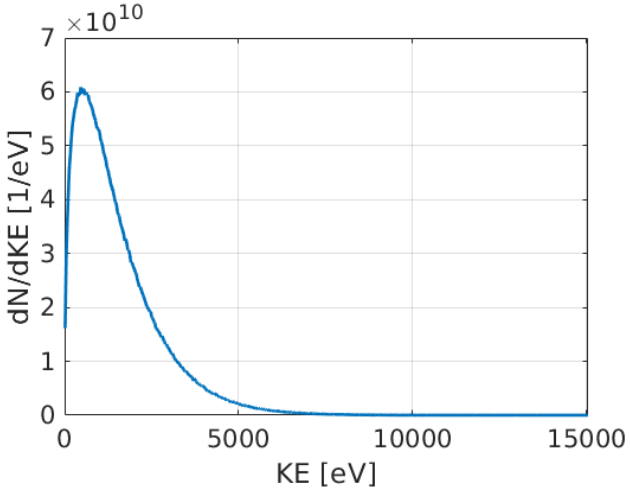
Energy [MeV]

UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 3.3: 2D particle loading

- Load a basic ionised carbon target
  - Maxwellian momentum distribution
  - Thin foil target shape
- Can plot either:
  - Grid-averaged quantities (density, temperature)
  - Particle properties (to get energy spectra)
  - Can also ignore some species for particle dumps

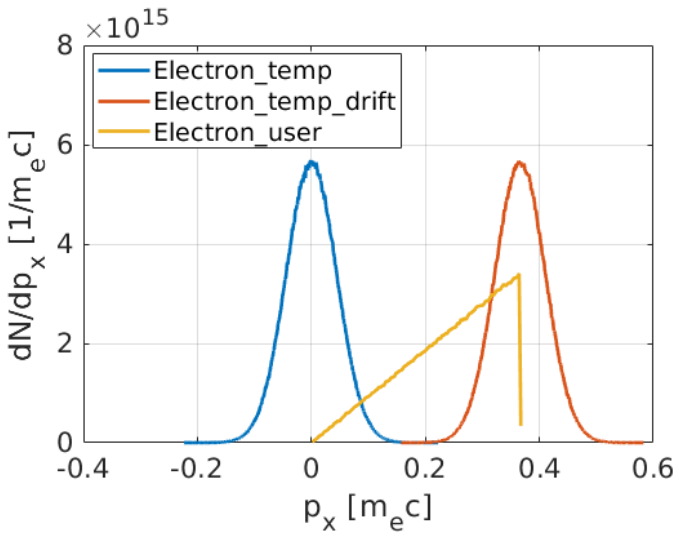





```
begin:control
    nx = 500
    ny = 500
    t_end = 1.0e-15
    x_min = 0
    x_max = 25e-6
    y_min = 0
    y_max = 25e-6
    stdout_frequency = 100
    npart = 50 * (5.0e-6/(x_max-x_min)) * nx * ny
end:control

begin:boundaries
    bc_x_min = open
    bc_x_max = open
    bc_y_min = open
    bc_y_max = open
end:boundaries

begin:species
    name = Electron
    mass = 1.0
    charge = -1.0
    frac = 0.8
    density = if (x gt 15.0e-6, 1.0e24, 0)
    density = if (x gt 20.0e-6, 0, density(Electron))
    temp_ev = 1000
end:species

begin:species
    name = Carbon
    mass = 22033.0
    charge = 6.0
    frac = 0.2
    density = density(Electron) / 6
    temp_ev = 1000
    dump = F       # Ignore this species in particle-dump
end:species

begin:output
    dt_snapshot = t_end
    number_density = always + species
    px = always
    py = always
    pz = always
    weight = always
end:output
```


UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 3.4: Momentum distributions



- EPOCH can load:
  - Maxwellian (default)
  - Maxwell-Juttner (relativistic Maxwellian)
  - User-defined distribution
  - Any of the above with a drift momentum
- User-defined method:
  - Provide a function $f(p_x, p_y, p_z)$ and momentum range
  - EPOCH uniformly samples $p_x,\ p_y,\ p_z$
  - Assuming f has maximum value of 1, calculate f
  - Rejection sampling: keep $p_x,\ p_y,\ p_z$ with probability f

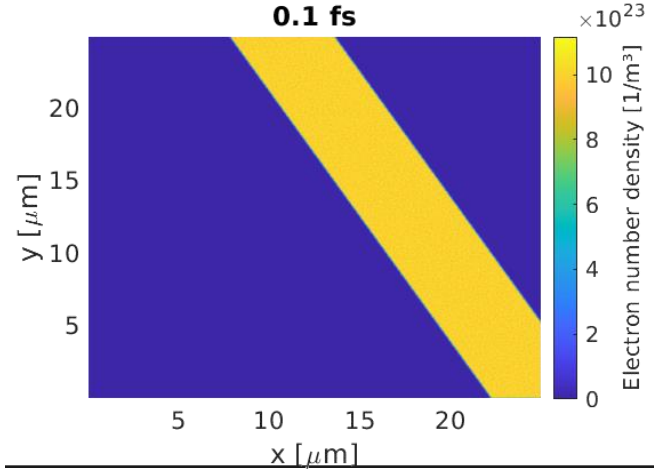
```
begin:species
    name = Electron_temp
    mass = 1.0
    charge = -1.0
    npart = 5 * nx * ny
    density = 1.0e24
    temp_ev = 1000
end:species

begin:species
    name = Electron_temp_drift
    mass = 1.0
    charge = -1.0
    npart = 5 * nx * ny
    density = 1.0e24
    temp_ev = 1000
    drift_x = 1.0e-22
end:species

begin:species
    name = Electron_user
    mass = 1.0
    charge = -1.0
    npart = 5 * nx * ny
    density = 1.0e24
    dist_fn = px / 1.0e-22
    dist_fn_px_range = (0, 1.0e-22)
end:species
```

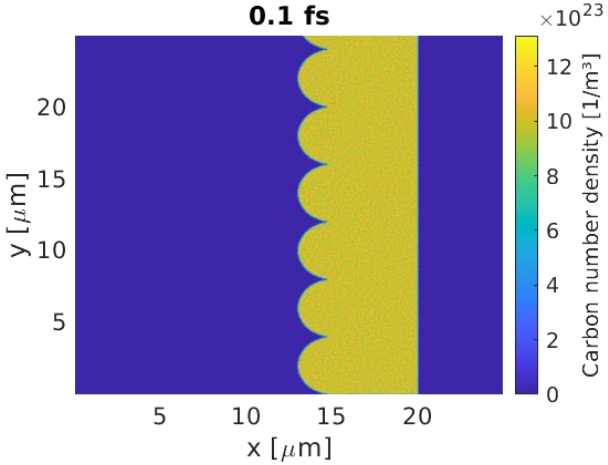

UK Ministry of Defence © Crown owned copyright 2026/AWE

Example 3.5: Funky targets

- Use the constant block to make functions
  - Makes it easier to make complex targets
  - Define rotated system for rotated targets
  - Define periodically repeating functions too

- Electron: x_rot, y_rot are rotated x and y
- Carbon: y_r is repeat distance, y_f is repeating fraction

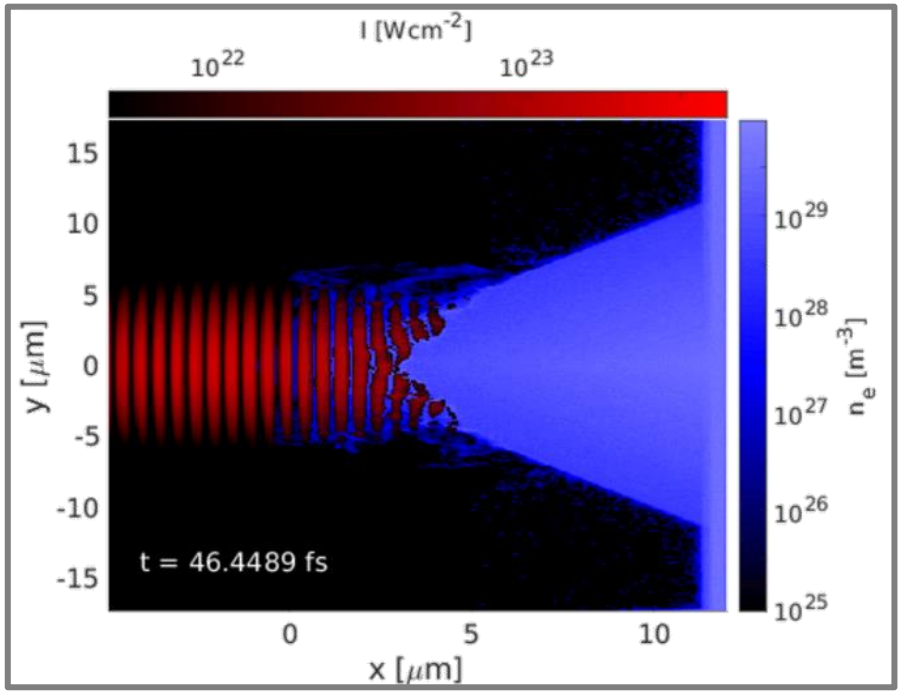





```
begin:constant
    x0 = 15.0e-6
    y0 = 12.5e-6
    theta = 30.0 / 180.0 * pi
    x_rot =  (x-x0)*cos(theta) + (y-y0)*sin(theta)
    y_rot = -(x-x0)*sin(theta) + (y-y0)*cos(theta)
end:constant

begin:species
    name = Electron
    mass = 1.0
    charge = -1.0
    frac = 0.8
    density = if (x_rot gt 0.0, 1.0e24, 0)
    density = if (x_rot gt 5.0e-6, 0, density(Electron))
    temp_ev = 1000
end:species

begin:constant
    x_front = 15.0e-6
    circle_radius = 2.0e-6
    y_r = 2.0 * circle_radius
    y_f = 2.0*((y / y_r) - floor(y / y_r))-1.0
end:constant

begin:species
    name = Carbon
    mass = 22033.0
    charge = 6.0
    frac = 0.2
    density = if ((x-x_front)^2 + (y_f * circle_radius)^2 lt circle_radius^2,\
                  1.0e24, 0)
    density = if (x gt x_front and x lt (x_front + 5.0e-6), \
                  1.0e24, density(Carbon))
    temp_ev = 1000
end:species
```


UK Ministry of Defence © Crown owned copyright 2026/AWE

Running laser-matter simulations



- Inject a laser into a simulation with a target
  - Ensure cell-size/particle number minimises self-heating
- Pre-plasma:
  - Add exponentially decaying plasma to target front
  - Mimics effects of laser pre-heating
- Convergence testing:
  - Once you have a final result, try doubling or halving the resolution
  - If the result doesn’t change by much, result has converged
  - Physical results don’t depend on numerical parameters!

UK Ministry of Defence © Crown owned copyright 2026/AWE

NUCLEAR SECURITY TECHNOLOGIES

www.awe.co.uk

UK Ministry of Defence © Crown owned copyright 2026/AWE

AWE Nuclear Security Technologies is the trading name for AWE plc. AWE is a Non-Departmental Public Body owned by the Ministry of Defence. Registered in England and Wales. Registration no. 02763902

AWE plc Aldermaston, Reading, Berkshire, RG7 4PR